Day(43):-

* Number of Approaches to define Functions:-

#Approach (1).

#program for defining a function for cal mul of two numbers.

#Input ---> Taking from function call

#process ---> Done in function Body.

#Result or output ---> Given to function call.

```
def mulop(a,b): # Here a,b are called formal parameters
        c=a*b   # Here c is called local variables
        return c
#main program
a=float(input("Enter first value:"))
b=float(input("Enter second value:"))
res=mulop(a,b)   # function call
print("Mul({},{})={}".format(a,b,res))
```

#Approach②.
#input - - -> Taking in function Body.
#process - - - -> Done in function Body.
#Result or output --> Displayed in function Body.

```
def mulop():
    #input
    a=float(input("Enter first value:"))
    b=float(input("Enter second value:"))
    #process
    c=a*b
    #result
    print("mul({},{})={}".format(a,b,c))

#main program.
mulop()  # function call
```

a,b.

#Approach③:-
#input - - - - -> Taking from Function call
#process - - - -> Done in function Body,
#Result or output --> Displayed in function Body.

```
def mulop(k,v):
    r=k*v
    print("mul({},{})={}".format(k,v,r))

#main program
a=float(input("Enter First value:"))
b=float(input("Enter second value:"))
mulop(a,b)  # function call
```

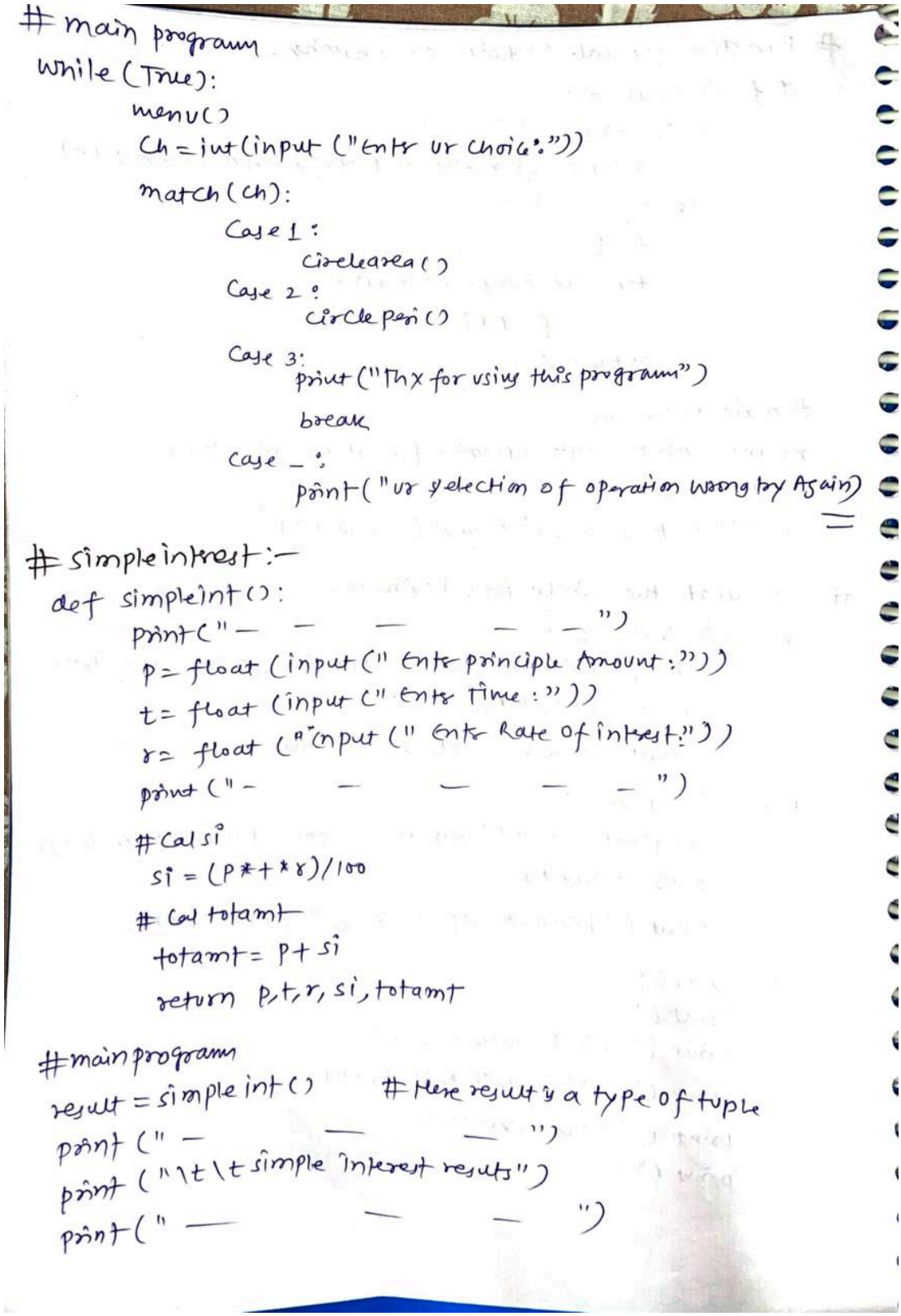
Approach ④ :-

#input - - - -> Taking in function Body
#process - - -> Done in function Body.
#Result or output - - -> Given to function call.

```
def mulop():
    #input
    a=float(input("Enter first value:"))
    b=float(input("Enter second value:"))
    #process
    c=a*b
    #Give the result to function call
    return a,b,c

#main program
a,b,res=mulop()  # function call with multipleline assignment
print("mul({},{})={}".format(a,b,res))
print("-------OR-------")
res=mulop()  # function call with single line assignment
#Here res is type of tuple.
print(res,type(res)) #(7.0,8.0,56.0) <class 'tuple'>
print("mul({},{})={}".format(res[-3],res[-2],res[-1]))
```

=.

```
# Function for cal. factorial of a number.
def factcal(n):
    if(n<0):
        return "{} is -ve and No Factorial".format(n)
    else:
        f=1
        for i in range(1,n+1):
            f=f*i
        return f

#main program
n=int(input("Entr a Number for cal factorial:"))
result=factcal(n)
print("fact({})={}".format(n,result))

# Calculate the circle Area Perimetr.
def circlearea():
    r=float(input("Enter radius for cal Area of circle"))
    ac=3.14*r**2
    print("Area of circle:{}".format(ac))
def circleperi():
    r=float(input("Entr radius for cal Peri. of circle"))
    pc=2*3.14*r
    print("perimetr of circle:{}".format(pc))
def menu():
    print("- - - - - -")
    print("\t\t1. circle Area:")
    print("\t\t2. circle perimeter:")
    print("\t\t0. Exit:")
    print("- - - - - -")
```



```
# main program
while (True):
    menu()
    ch = int(input("Enter ur choice:"))
    match(ch):
        case 1:
            circlearea()
        case 2:
            circleperi()
        case 3:
            print("Thx for using this program")
            break
        case _:
            print("ur selection of operation wrong try Again")

# simple interest :-
def simpleint():
    print("-  -  -  -  -")
    p = float(input("Enter principle Amount:"))
    t = float(input("Enter Time:"))
    r = float(input("Enter Rate of interest:"))
    print("-  -  -  -  -")
    # Cal si
    si = (p*t*r)/100
    # Cal totamt
    totamt = p+si
    return p,t,r,si,totamt

# main program
result = simpleint()    # Here result is a type of tuple
print("-  -  -")
print("\t\t simple interest results")
print("-  -  -")
```

Approach 1:

program for defining a function for calculate multiply of two numbers

In [1]:
```
#input---->>taking from function call
#process--->>done in function body
#result or output----->>given to function call
def mulop(a,b):
    c=a*b
```

```
    return c
#main program
a=float(input("enter first value"))
b=float(input("enter second value"))
res=mulop(a,b)
print("mul ({},{})={}".format(a,b,res))
```

```
mul (10.0,8.0)=80.0
```

Approach 2:

In [7]:
```
#input====>>taking in function body
#process===>>done in function body
#result or output====>> displayed in function body
def mulop(a,b):
    #input
    a=float(input("enter a first value:"))
    b=float(input("enter a second value:"))
    #process
    c=a*b
    #result
    print("mul ({},{})={}".format(a,b,c))
#main program
mulop(a,b)
```

```
mul (12.0,8.0)=96.0
```

Approach 3:

In [10]:
```
#input------>taking from function call
#process---->>done in function body
#output----->>>displayed in function body
def mulop(k,v):
    r=k*v
    print("mul({},{})={}".format(k,v,r))
#main program
a=float(input("enter a first value:"))
b=float(input("enter a send value:"))
mulop(a,b)
```

```
mul(21.0,8.0)=168.0
```

Approach 4:

In [15]:
```
#input----->> taking in function body
#process--->> done in function body
#result---->>given to function call
def mulop():
    a=float(input("enter a first value:"))#input
    b=float(input("enter a second value:"))
    c=a*b#pprocess
    return a,b,c
#main program
a,b,res=mulop()
print("mul({},{})={}".format(a,b,res))
print("===========OR=======")
res=mulop()
```

```
print(res,type(res))
print("mul({},{})={}".format(res[-3],res[-2],res[-1]))
```

```
mul(12.0,8.0)=96.0
==========OR=======
(12.0, 8.0, 96.0) <class 'tuple'>
mul(12.0,8.0)=96.0
```

function for calculate factorial of a :number

In [19]:
```
def factcal(n):
    if (n<0):
        return "{} is -ve and NO factorial".format(n)
    else:
        f=1
        for i in range (1,n+1):
            f=f*i
        return f
#main program
n=int(input("enter anumber for calculate factorial:"))
result=factcal(n)
print("fact({}!)={}".format(n,result))
```

```
fact(4!)=24
```

calculate the circle Area perimeter

In [ ]:
```
def circlearea():
    r=float(input("enter radius for calculate area of circle"))
    ac=3.14*r**2
    print("area of circle:{}".format(ac))
def circleperi():
    r=float(input("enter radius for calculate perimeter of circle"))
    pc=2*3.14*r
    print("perimeter of circle:{}".format(pc))
def menu():
    print("="*100)
    print("\t\t1.circle Area:")
    print("\t\t2.circle perimeter:")
    print("\t\t3.Exit:")
    print("="*100)
#main program
while(True):
    menu()
    ch=int(input("enter ur choice:"))
    match (ch):
        case 1:
            circlearea()
        case 2:
            circleperi()
        case 3:
            print("thanks for using this program")
            break
        case _:
            print("ur selection operation is wrong 'TRY AGAIN'")
```

```
==========================================================================================
==================
                1.circle Area:
                2.circle perimeter:
                3.Exit:
==========================================================================================
==================
perimeter of circle:75.36
==========================================================================================
==================
                1.circle Area:
                2.circle perimeter:
                3.Exit:
==========================================================================================
==================
```

simple interest

In [8]:
```python
def simpleint():
    print("="*50)
    p=float(input("enter principle amount:"))
    t=float(input("enter time:"))
    r=float(input("enter rate of interest"))
    print("="*50)
    si=(p*t*r)/100
    totamnt=p+si
    return p,t,r,si,totamnt
#main program
result=simpleint()
print("="*50)
print("\t\t simple interest results:")
print("\t principle amount:{}".format(result[0]))
print("\t Time:{}".format(result[1]))
print("\t Rate of Interest:{}".format(result[2]))
print("\t simple interest:{}".format(result[3]))
print("\t Total amount to pay:{}".format(result[4]))
```

```
==================================================
==================================================
==================================================
                 simple interest results:
         principle amount:2.0
         Time:4.0
         Rate of Interest:2.0
         simple interest:0.16
         Total amount to pay:2.16
```

In [ ]:

In [ ]: